Cold start latency in AWS Lambda: does a lightweight ping help?

Arnav Chhabra · BITS Pilani · ID: 2023A7PS0567P · Track D (serverless performance benchmark) · Random seed: 67

1. Goal

My goal was to understand how a lightweight pre warming strategy affects request latency and cold start frequency in AWS Lambda.

A cold start is the delay that occurs when a serverless function is invoked but no runtime environment is available to run it immediately. AWS Lambda is an ephemeral compute service it creates containers or micro virtual machines on demand and destroys them after a period of inactivity, to save cost and resources. This matters for user facing services because the delay is added to the request, in my tests it was about 500 ms.

To study this, I created a tiny function and tested two strategies. One function was called every 6 minutes, longer than an environment survives, so every call was a cold start. The other, an identical function, was kept warm by a scheduled ping every 3 minutes. Comparing the two shows how much time pre warming saves.

2. Method

I used AWS Lambda because I was already familiar with it. The function is written in Python 3.12 (128 MB, region ap-south-1) and is deliberately small, so the measured time is mostly startup time. The limitation is that a larger function would likely have a higher coldstart latency, which I did not test. AWS logs show an Init Duration only when a call is a cold start, which let me tell cold calls from warm ones.

Pilot. AWS does not publish how long an idle environment survives, so I measured it first. I created five functions with the same code and called each after a different idle gap: 5, 10, 20, 30 and 45 minutes, with 4 repeats planned for each. The run was interrupted by a network outage, but the 5 and 10 minute gaps had finished all repeats the 5 minute calls were warm every time and the 10 minute calls were cold every time. So my initial assumption that an environment stays alive for a long time was wrong, and the limit lay between 5 and 10 minutes. I then tested gaps of 6, 7, 8 and 9 minutes, and every call was cold. The environment therefore dies somewhere between the 5th and 6th minute, and I used 6 minutes as the request gap, the shortest gap that was cold every time.

Ping interval. The scheduled ping has to arrive before the environment dies, so it must be well under 5 minutes. I chose 3 minutes enough margin before the environment dies, without pinging so often that it stops being lightweight. It is offset by one minute from the requests so the two never arrive at the same moment.

Main run. I created two functions with the same code. The baseline was called every 6 minutes, so every call was a cold start. The ping function was kept warm by the 3 minute scheduled ping. Both were called at the same moment every 6 minutes, so that time of day and network conditions affect both equally, for 30 rounds. For each call I recorded cold or warm (from Init Duration), the server side latency (init time plus run time) and the laptop side time. I report median, p95 and cold start rate, with server side latency as the main measure because laptop side timing includes network noise. The only randomness is the order in which the two calls are sent within a round, shuffled with seed 67 (the last two digits of my ID).

Burst test. To check whether a ping helps when several requests arrive at once, I sent simultaneous requests to a function that had one warm environment.

3. Results

Measured results from 30 rounds (server side latency = Init Duration for cold starts plus handler Duration):

	Baseline (no ping)	Ping every 3 min
Requests	30	30
Cold starts	30 (100%)	0 (0%)
Median latency	496.8 ms	30.5 ms
p95 latency	713.2 ms	57.1 ms
Max latency	1013.7 ms	58.8 ms

Every baseline call was a cold start, and its median of 496.8 ms is almost all Init Duration (median 495.0 ms). The pinged function had no cold starts, with a median of 30.5 ms, about 16 times lower. With only 30 requests per strategy, p95 is close to the maximum and should be read as rough. One unexpected result: the pinged function's warm latency was about 30 ms (roughly 2 to 59 ms), higher than the ~2 ms I saw in earlier tests; I did not find the cause.

Burst test. With a 2 ms handler, 5 simultaneous requests produced 0 cold starts, because each call finished before the next arrived. With a handler that sleeps 1 second, starting from one warm environment, the 5 requests ran in 5 separate environments, giving 4 cold starts in four of five rounds (round 0 had 4 environments and 3 cold starts because one request arrived late). So one warm environment does not cover overlapping requests.

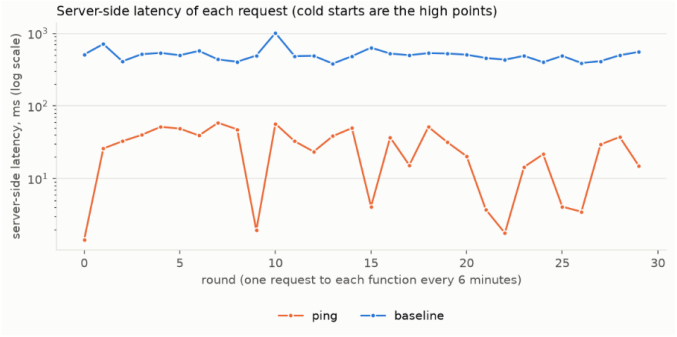


Figure 1: Server side latency of each request, log scale. The blue line (baseline) is cold every round; the orange line (ping) is warm.

4. Limitations

Main limitation: traffic pattern. I tested only one traffic pattern, a request every 6 minutes, which is longer than the environment's lifetime, so the baseline was cold on every call. If requests had arrived more often than every 5 minutes, the baseline would have stayed warm on its own and the scheduled ping would have been useless. So my result, that pre warming removes cold starts, holds only for sparse traffic. To test this I would repeat the experiment with gaps of 1, 3, 5 and 10 minutes and find where the ping stops making a difference.

Language and function size. I used only Python, at 128 MB, with a very small function. Different languages are generally reported to have different cold start behaviour interpreted languages such as Python and aheadoftime compiled languages such as Go or Rust usually start faster than runtimes that rely on justintime (JIT) compilation, such as Java or C#. I could not measure this. Real applications also have many large dependencies and libraries that add to the cold start latency, so my ~500 ms is probably a best case.

Other limitations. A ping keeps only one environment warm, so overlapping requests still hit cold starts (burst test). The experiment is small and short 30 requests per strategy, one region, one afternoon, and AWS does not document the idle timeout. The 6 minute gap is just past the observed 5 minute edge. Provisioned concurrency was not tested because my account's concurrency limit blocked it. The 30 ms warm latency of the ping function is unexplained. All timing was done from one laptop on a hotspot, so I used server side latency as the main measure.

5. Where I got stuck

Waiting time. I assumed an idle environment would survive tens of minutes and planned 40 minute gaps, which would take about 20 hours for 30 samples. A pilot showed it lasts only about 5 minutes, so I used a 6 minute gap and ran both functions side by side.

Burst test. My first test gave 0 cold starts out of 5, not the 4 I expected. The 2 ms function finished before the next request arrived, so one environment was reused. I rebuilt the handler to sleep 1 second and return its environment ID, and then the 5 requests used 5 environments in 4 of 5 rounds. So cold starts come from overlapping requests, not from the number of requests.

Unresolved. The ping function's warm latency was about 30 ms instead of the 2 ms I saw earlier, and I did not find the cause.

6. Week two plan

In week two I would first make the function more realistic: a larger function with heavier dependencies, to see how cold start grows with size, and the same function in other languages such as Node.js, Java and Go, to test the language claim in my limitations. I would then test a mitigation: moving rarely used imports out of the startup code and loading them only when needed (lazy loading), and measure how much Init Duration drops. Finally I would repeat the main run with request gaps of 1, 3, 5 and 10 minutes to find where the ping stops helping.

AI use disclosure

I used Claude (Anthropic) to explain Lambda concepts, review my experiment design, and help write and debug the scripts and AWS commands. I ran all experiments and chose the gaps and intervals. I wrote the report myself , Claude corrected factual errors and edited the wording, and the text here is that edited version.